МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА

ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

Лабораторна робота № 2

з дисципліни **«Технології створення програмних продуктів»**

Тема: «Опис програмної реалізації системи»

Виконав:

студент гр. ІР-31

Петро Кравченко

Перевірив:

Зараховано від

Кравченко Ольга Віталіївна

_____

(підпис викладача)

Київ-2026

ЗМІСТ

1. МЕТА ТА ЗАВДАННЯ РОБОТИ 3
2. МЕТА ПРОГРАМНОГО ПРОДУКТУ 4
3. ВХІДНІ ТА ВИХІДНІ ДАНІ 5
4. СИСТЕМНІ ВИМОГИ 7
5. ПРОГРАМНІ ЗАСОБИ РЕАЛІЗАЦІЇ ТА ОБҐРУНТУВАННЯ ВИБОРУ 9
5.1. СКБД PostgreSQL 9
5.2. Python і Flask 10
5.3. Засоби автоматизації та перевірки 10
6. АЛГОРИТМ РОБОТИ ПРОГРАМНОГО ПРОДУКТУ 11
6.1. Структура програмного продукту 11
6.2. Загальний алгоритм обробки запиту 12
6.3. Модуль прийому виявлень 14
6.4. Модуль керування місією 15
6.5. Модуль журналу аудиту 17
7. ОПИС ІНТЕРФЕЙСУ ПРОГРАМНОГО ПРОДУКТУ 19
7.1. Вхід до системи 19
7.2. Повітряна обстановка 19
7.3. Життєвий цикл місії 21
7.4. Ресурси 26
7.5. Аналітика та журнал аудиту 29
7.6. Програмний інтерфейс для шлюзів 31
8. РЕАЛІЗАЦІЯ ФУНКЦІОНАЛЬНИХ ВИМОГ ДО БАЗИ ДАНИХ 33
ВИСНОВКИ 35
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 36

1. МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи: описати програмну реалізацію системи — мету програмного продукту, вхідні й вихідні дані, системні вимоги, програмні засоби реалізації, алгоритм роботи основних модулів та інтерфейс — і продемонструвати реалізацію функціональних вимог до бази даних, сформульованих у лабораторній роботі 1.

Завдання роботи [1]:

1) чітко сформулювати мету програмного продукту — розділ 2;

2) виписати вхідні та вихідні дані програмного продукту — розділ 3;

3) виписати системні вимоги до програмного забезпечення, що забезпечать злагоджену роботу програми — розділ 4;

4) коротко описати програмні засоби реалізації програмного продукту з обґрунтуванням технічною мовою, чому їх обрано, — розділ 5;

5) описати алгоритм роботи програмного продукту, зобразити блок-схеми роботи основних модулів — розділ 6;

6) описати інтерфейс програмного продукту — розділ 7.

Додатково в розділі 8 показано, як реалізовано та перевірено функціональні вимоги ФВ-БД-01 — ФВ-БД-13 до програмної реалізації бази даних, виписані у висновках лабораторної роботи 1 [2]. Структура опису відповідає розділу «Опис програмної реалізації» кваліфікаційних робіт кафедри [3]. Звіт оформлено відповідно до ДСТУ 3008:2015 [4].

Примітка: РЦПДО «Щит-Північ» і IoT-платформа «Щит-Лінк» є навчальними умовними моделями; усі назви, дані та показники наведено суто з навчально-ілюстративною метою.

2. МЕТА ПРОГРАМНОГО ПРОДУКТУ

Програмний продукт — IoT-платформа «Щит-Лінк» на тему «Розподілена IoT-система цілевказівки та координації дронів для перехоплення повітряних цілей». Вона призначена для регіонального центру протидронової оборони, якому потрібно за лічені хвилини перетворити потік виявлень від розподіленої сенсорної мережі на обґрунтоване рішення: яку ціль, яким розрахунком і яким перехоплювачем уражати, — і при цьому зберегти повний, незмінний слід кожного рішення.

Мета програмного продукту — скоротити час від виявлення повітряної цілі до призначення розрахунку й підвищити частку успішних перехоплень за рахунок автоматизації: приймання та зіставлення виявлень у треки, оцінки загрози з урахуванням об’єктів прикриття, підбору готового розрахунку, контрольованого життєвого циклу бойового завдання з правилом двох осіб і незмінного журналу аудиту.

Реалізована версія продукту охоплює ядро системи: базу даних PostgreSQL з серверною логікою, веб-застосунок автоматизованих робочих місць (АРМ) для п’яти груп користувачів і REST API для периферійних шлюзів. Вона безпосередньо реалізує вимоги до програмної реалізації БД з лабораторної роботи 1 і функціональні вимоги практичної роботи № 1 щодо формування обстановки, призначення та санкціонування місій, обліку ресурсів, аналітики й аудиту [5]. Функції, що потребують реального обладнання (обмін із сенсорами за MQTT, телеметрія MAVLink), подано програмним імітатором шлюзу та тестовими даними.

Очікувані результати застосування: кожне виявлення потрапляє в трек протягом однієї транзакції; черга загроз упорядковується автоматично; жодна місія не може бути виконана без санкції іншої особи; будь-яку зміну бойових даних можна відстежити й перевірити на цілісність; керівник отримує показники ефективності за довільний період.

3. ВХІДНІ ТА ВИХІДНІ ДАНІ

Вхідні дані надходять від двох груп джерел: автоматично — від периферійних шлюзів сенсорних вузлів і наземних станцій розрахунків, та вручну — від користувачів через форми АРМ. Перелік вхідних даних наведено в табл. 2.1; кожен вид даних зберігається у відповідних таблицях фізичної моделі ЛР1.

Таблиця 2.1 — Вхідні дані програмного продукту

Джерело	Дані	Канал і формат	Таблиці БД
Периферійний шлюз	Виявлення: заводський номер сенсора, час, широта, довгота, висота, швидкість, пеленг, клас сигнатури, достовірність	REST API, JSON, ключ шлюзу	detection, air_track, sensor
Наземна станція розрахунку	Кадри телеметрії: час, координати, висота, швидкість, заряд батареї, якість зв’язку	Пакетне завантаження (імітатор)	telemetry_frame
Аналітик обстановки	Класифікація треку: належність, тип цілі	Форма картки треку	air_track
Офіцер бойового управління	Пропозиція місії: трек, розрахунок; перепризначення	Форма картки треку, картка місії	mission, mission_event
Оперативний черговий	Санкція або відхилення місії	Кнопки картки місії	mission
Командир розрахунку	Прийняття завдання, пуск із вибором перехоплювача, доповідь про результат, причина скасування	Картка місії	mission, engagement_report
Логіст, технік	Облік перехоплювачів: серійний номер, модель, позиція, стан, прошивка	Форма перехоплювача	interceptor
Усі користувачі	Вхід (обліковий запис); період аналізу; фільтри	Форми АРМ	personnel, параметри запитів

Вихідні дані — інформація, яку продукт надає користувачам і зовнішнім системам (табл. 2.2). Частину з них обчислює СКБД (представлення, функції), частину формує застосунок.

Таблиця 2.2 — Вихідні дані програмного продукту

Вихідні дані	Зміст	Отримувач	Джерело в БД
Черга загроз	Активні треки за спаданням оцінки загрози: стан, належність, час підльоту, об’єкт під загрозою, активна місія	Аналітик, ОБУ, ОЧ; зовнішні системи (JSON)	v_threat_queue
Картка треку	Остання позиція, швидкість, класифікація, останні виявлення, кандидати-розрахунки з відстанню	Аналітик, ОБУ	fn_track_pos, fn_crew_candidates
Картка місії	Учасники, часові мітки, перехоплювач, результат, хронологія подій, телеметрія, доступні дії	ОБУ, ОЧ, розрахунок	mission, mission_event, telemetry_frame
Готовність ресурсів	За позиціями: розрахунки готові / на завданні, готові перехоплювачі	Логіст, ОЧ, начальник центру	v_readiness
Показники ефективності	Кількість місій, влучань, промахів, скасувань; частка влучань; середній час «пропозиція — санкція», «санкція — пуск»; рейтинг розрахунків	Аналітик ефективності, начальник центру	fn_mission_stats
Журнал аудиту	Записи змін з хешами SHA-256, результат перевірки цілісності ланцюжка	Офіцер кібербезпеки	audit_log, fn_audit_verify
Відповіді API	201 — номер треку; 401 — невірний ключ шлюзу; 422 — причина відмови	Периферійний шлюз	fn_ingest_detection
Повідомлення	Підтвердження дій і причини відмов (порушення правил БД)	Користувач АРМ	винятки функцій і тригерів

4. СИСТЕМНІ ВИМОГИ

Вимоги до програмного забезпечення поділяють на функціональні, нефункціональні та бізнес-вимоги [6]; системні вимоги належать до нефункціональних і визначають апаратне й програмне середовище, у якому продукт працює злагоджено. Вимоги сформульовано для трьох вузлів: сервера застосунку й СКБД, АРМ користувача та периферійного шлюзу (табл. 2.3). Вони узгоджені з нефункціональними вимогами практичної роботи № 1 [5]: затримка «сенсор — екран АРМ» до 2 с (NFR-01), доставка завдання розрахунку до 1 с (NFR-02), двофакторний вхід, журналювання всіх дій. Наведені вимоги стосуються реалізованого ядра для одного центру; цільова конфігурація з резервуванням і шиною подій описана в технічних вимогах SRS практичної роботи № 1.

Таблиця 2.3 — Системні вимоги до програмного забезпечення

Вузол	Апаратні вимоги	Програмні вимоги
Сервер (застосунок і СКБД)	Процесор від 4 ядер, 2,5 ГГц; оперативна пам’ять від 8 ГБ; SSD від 100 ГБ (журнали виявлень і телеметрії ростуть приблизно на 1 ГБ на місяць); два мережеві інтерфейси	ОС Linux (64 біт); PostgreSQL 13 або новіший (перевірено на 18.6); Python 3.11 або новіший (перевірено на 3.14); Flask 3.1.1, Jinja2 3.1.6, psycopg 3.2.10; зворотний проксі з TLS (nginx)
АРМ користувача	Будь-який ПК або ноутбук з двоядерним процесором і 4 ГБ пам’яті; екран від 1280 × 720 (інтерфейс працює від ширини 720 пікселів)	Сучасний браузер (Chrome, Firefox, Edge актуальних версій) з увімкненими cookie; встановлення ПЗ не потрібне
Периферійний шлюз	NVIDIA Jetson Orin Nano Super, обрана в практичній роботі № 3; для самого передавання виявлень через REST API достатньо будь-якого одноплатного комп’ютера	Клієнт HTTP(S) з підтримкою JSON; ключ шлюзу X-Gateway-Token
Мережа	Локальна мережа центру від 100 Мбіт/с; резервний канал до позицій	HTTPS між АРМ, шлюзами й сервером; доступ до СКБД лише із сервера застосунку

Достатність вимог перевірено на тестовому стенді (комп’ютер з 18 логічними ядрами, PostgreSQL 18.6): наповнення БД тестовими даними за 30 діб (651 виявлення, 30 місій, 282 записи аудиту) триває 0,3 с; обробка одного виявлення через REST API займає в медіані 7,8 мс (95-й процентиль — 9,8 мс,

максимум — 9,8 мс), тобто становить менше 1 % бюджету затримки NFR-01. Повна резервна копія БД має обсяг 174 КБ.

5. ПРОГРАМНІ ЗАСОБИ РЕАЛІЗАЦІЇ ТА ОБҐРУНТУВАННЯ ВИБОРУ

Мову, фреймворк і СКБД обрано за вимогами до продукту (підрозділи 5.1–5.2), а допоміжні засоби — за категоріями інструментів лекції 7: керування версіями, автоматизації процесів і тестування [7]. Критеріями були: надійність зберігання даних і виконання правил на рівні БД, мінімальна кількість залежностей, швидкість розроблення та можливість автоматичної перевірки. Перелік засобів наведено в табл. 2.4.

Таблиця 2.4 — Програмні засоби реалізації

Засіб	Версія	Призначення в продукті
PostgreSQL	18.6	СКБД: таблиці, обмеження, функції PL/pgSQL, тригери, представлення, ролі
Python	3.14	Мова серверної частини застосунку, скриптів наповнення й перевірки
Flask	3.1.1	Веб-фреймворк: маршрутизація, сеанси, обробка форм, REST API
Jinja2	3.1.6	Шаблонізатор HTML-сторінок АРМ
psycopg	3.2.10	Драйвер PostgreSQL для Python: транзакції, параметризовані запити
HTML5, CSS3	—	Розмітка й оформлення інтерфейсу без клієнтських фреймворків
Git	2.x	Керування версіями вихідного коду, моделей і звітів
Puppeteer, Chrome headless	25.12	Автоматизований прогін сценаріїв інтерфейсу і знімки екрана

5.1. СКБД PostgreSQL

PostgreSQL обрано ще на етапі фізичного моделювання [2]; на етапі реалізації вирішальними стали процедурна мова PL/pgSQL і тригери [8]. Усі критичні правила предметної області — допустимі переходи стану місії, правило двох осіб, узгодження станів ресурсів, журнал аудиту — виконуються всередині СКБД. Тому їх неможливо обійти ні через інтерфейс, ні через API, ні прямим SQL-запитом. Вбудована функція sha256 [9] дає змогу будувати ланцюжок хешів без зовнішніх бібліотек, оператор GRANT [10] — розмежувати доступ на рівні таблиць і стовпців, утиліти pg_dump і pg_restore [11] — виконувати резервне копіювання.

5.2. Python і Flask

Python забезпечує короткий і читабельний код та має зрілий драйвер PostgreSQL. Flask — мікрофреймворк, що надає лише маршрутизацію, сеанси й обробку запитів [12]; для продукту, де бізнес-логіка зосереджена в БД, цього достатньо, а відсутність ORM дає змогу викликати функції й представлення PostgreSQL напряму. Увесь веб-застосунок займає 291 рядок коду. Шаблонізатор Jinja2 [13] автоматично екранує дані, що захищає від міжсайтових сценаріїв, а psycopg [14] передає параметри окремо від тексту запиту, що виключає SQL-ін’єкції.

5.3. Засоби автоматизації та перевірки

Розгортання, наповнення, демонстрація і перевірка автоматизовані одним скриптом: він піднімає тимчасовий кластер PostgreSQL, виконує схему ЛР1 і серверну логіку, наповнює БД, запускає застосунок, проходить сценарій інтерфейсу в браузері без графічного вікна за допомогою бібліотеки Puppeteer [15] і перевіряє кожну функціональну вимогу. Тому всі знімки екрана в розділі 7 отримано з працюючої програми, а результати в розділі 8 відтворюються одним запуском.

6. АЛГОРИТМ РОБОТИ ПРОГРАМНОГО ПРОДУКТУ

6.1. Структура програмного продукту

Продукт побудовано за багаторівневою архітектурою (Layered Architecture) з розподілом відповідальності за шаблоном MVC [16]: модель — це БД з таблицями, функціями та тригерами; подання — HTML-шаблони; контролер — обробники маршрутів Flask. Структуру показано на рис. 2.1. Користувачі звертаються до застосунку з браузера, периферійні шлюзи — до REST API; застосунок працює з СКБД через psycopg і в кожній транзакції передає ідентифікатор користувача, за яким тригери перевіряють права і ведуть аудит.

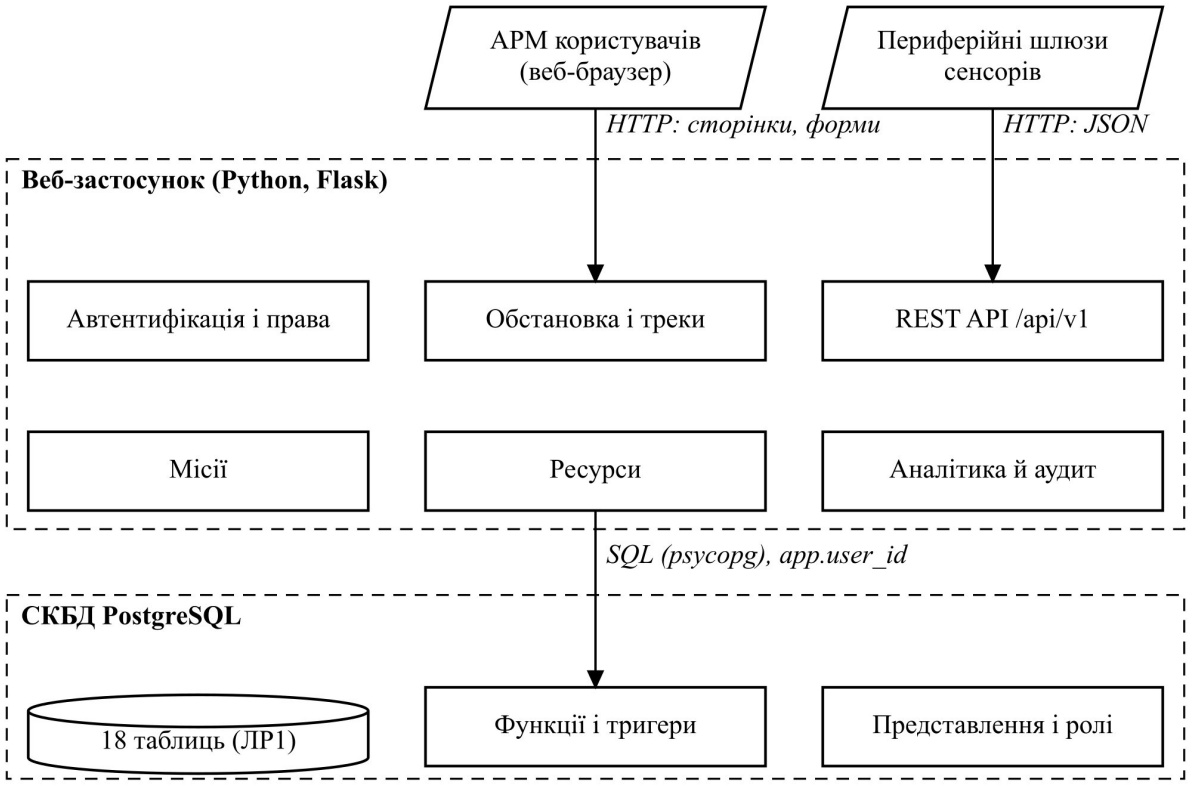


Рис. 2.1 — Структура програмного продукту

Склад модулів наведено в табл. 2.5. Реакцію БД на зміни реалізовано за подієвим принципом: тригер спрацьовує на подію зміни запису й виконує пов’язані дії, як спостерігач (Observer) у поведінкових шаблонах [16].

Таблиця 2.5 — Модулі програмного продукту

Модуль (файл)	Рядків	Призначення
db/impl.sql	340	Серверна логіка БД: 53 функції, 9 тригерів, 2 представлення, 5 ролей СКБД, таблиця переходів станів
db/seed.py	285	Наповнення БД тестовими даними через серверні функції
app/app.py	291	Веб-застосунок: автентифікація, обстановка, місії, ресурси, аналітика, аудит, REST API
app/templates/*.html	197	Десять шаблонів сторінок АРМ
app/static/style.css	47	Оформлення інтерфейсу
run_demo.py, pgtemp.py	289	Розгортання, демонстрація, перевірка вимог, резервне копіювання

6.2. Загальний алгоритм обробки запиту

Блок-схеми виконано символами ISO 5807 [17]: «термінатор» — початок і кінець, паралелограм — введення й виведення даних, прямокутник — процес, ромб — рішення, прямокутник з подвійними бічними лініями — визначений процес (підпрограма), коло — з’єднувач, що продовжує схему в іншому стовпці.

Загальний алгоритм (рис. 2.2) однаковий для всіх сторінок АРМ. Якщо сеанс не встановлено, користувач потрапляє на сторінку входу. Далі застосунок завантажує обліковий запис і права; за відсутності права на розділ повертається відповідь 403. Кожна дія виконується в окремій транзакції, на початку якої в СКБД передається ідентифікатор користувача. Якщо функція чи тригер БД відхиляє дію, транзакцію відкочують, а користувач бачить зрозуміле повідомлення про причину; інакше транзакцію фіксують і формують сторінку.

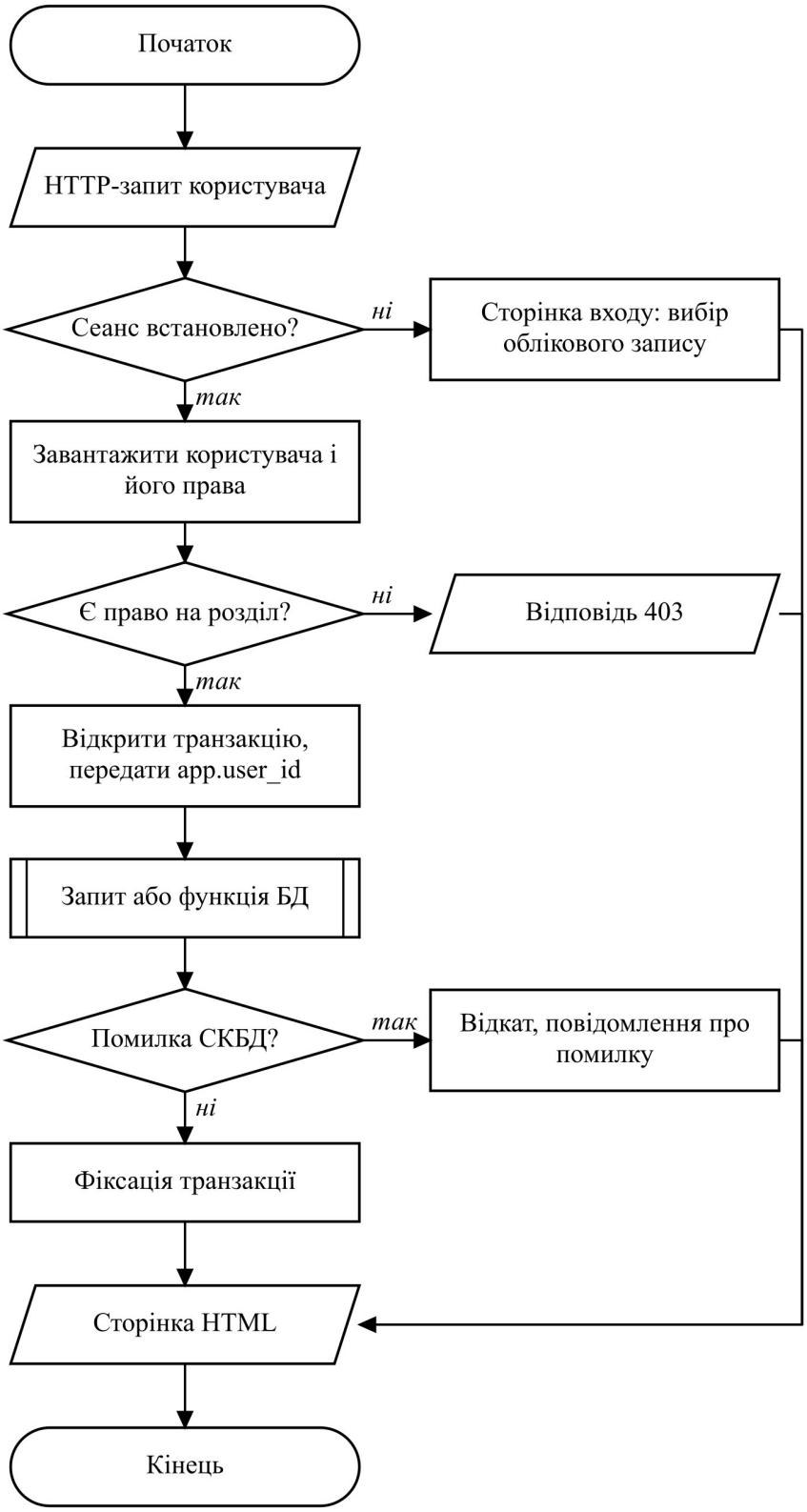


Рис. 2.2 — Блок-схема загального алгоритму обробки запиту користувача

6.3. Модуль прийому виявлень

Модуль прийому виявлень (рис. 2.3) — функція fn_ingest_detection, яку викликає REST API. Після перевірки ключа шлюзу функція перевіряє, що сенсор зареєстрований і перебуває в мережі, оновлює час його останнього сигналу. Виявлення з координатами зіставляється з найближчим активним треком, оновленим не раніше ніж 20 с тому, у радіусі 3 км; якщо такого немає, створюється новий попередній трек з номером ЩП-NNNN у межах доби. Після трьох виявлень від двох різних сенсорів трек стає «підтвердженим». Насамкінець підпрограма fn_eval_threat визначає найближчий об’єкт прикриття, час підльоту за поточною швидкістю й оцінку загрози з урахуванням категорії об’єкта.

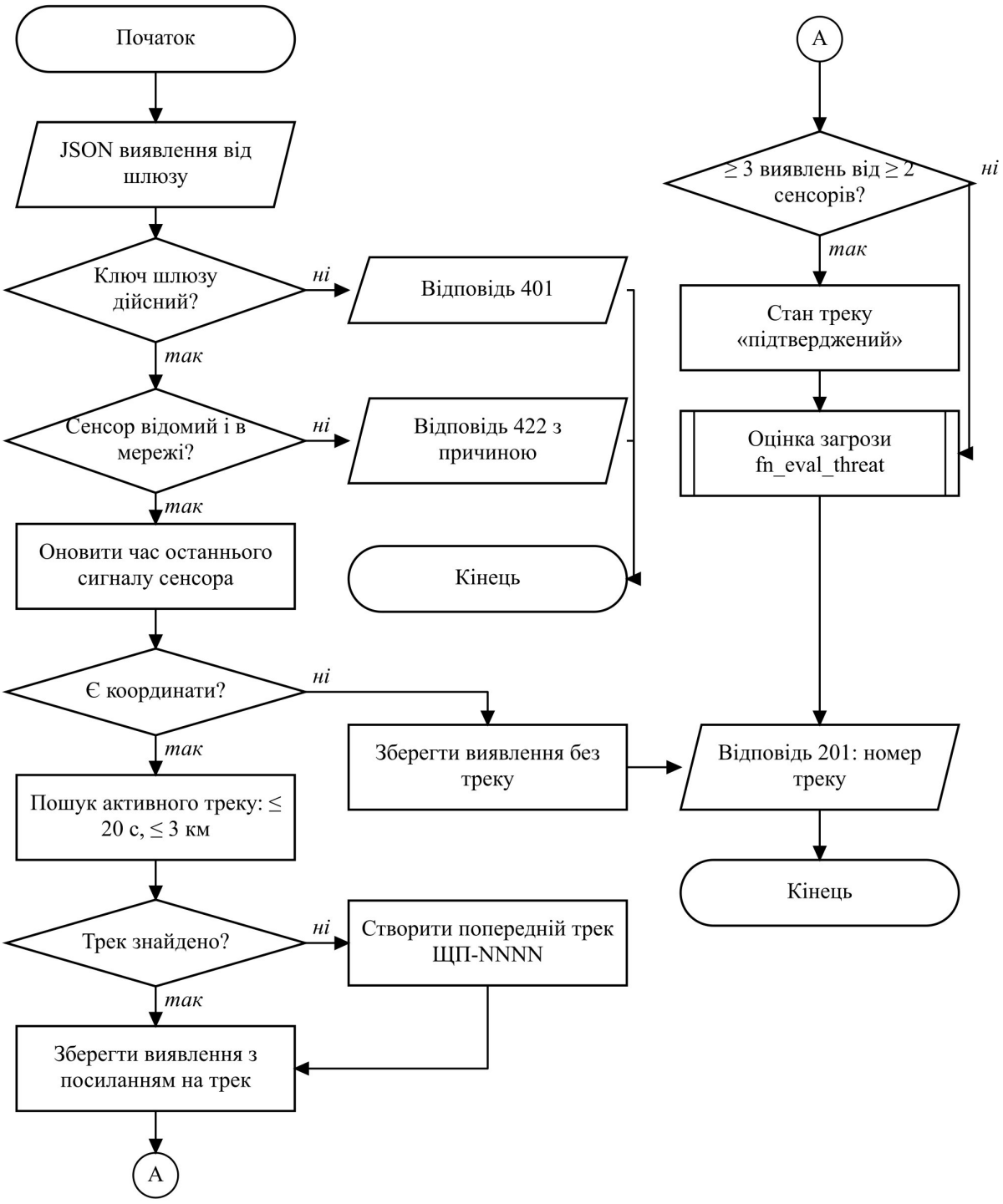


Рис. 2.3 — Блок-схема модуля прийому виявлень

6.4. Модуль керування місією

Модуль керування місією (рис. 2.4) реалізовано тригерами trg_mission_guard і trg_mission_after. Кожну зміну стану перевіряють за таблицею mission_transition, що містить вісім допустимих переходів діаграми

станів «Місії» з практичної роботи № 4 [18], і за правом користувача на перехід. Обмеження CHECK забороняє санкцію автором пропозиції (правило двох осіб), а перехід у стан «виконується» можливий лише з готовим перехоплювачем. Після успішної перевірки тригер записує часові мітки, оновлює стани розрахунку, перехоплювача й треку та додає подію в хронологію; будь-яка відмова скасовує всю транзакцію. Фрагмент тригерної функції наведено на рис. 2.5.

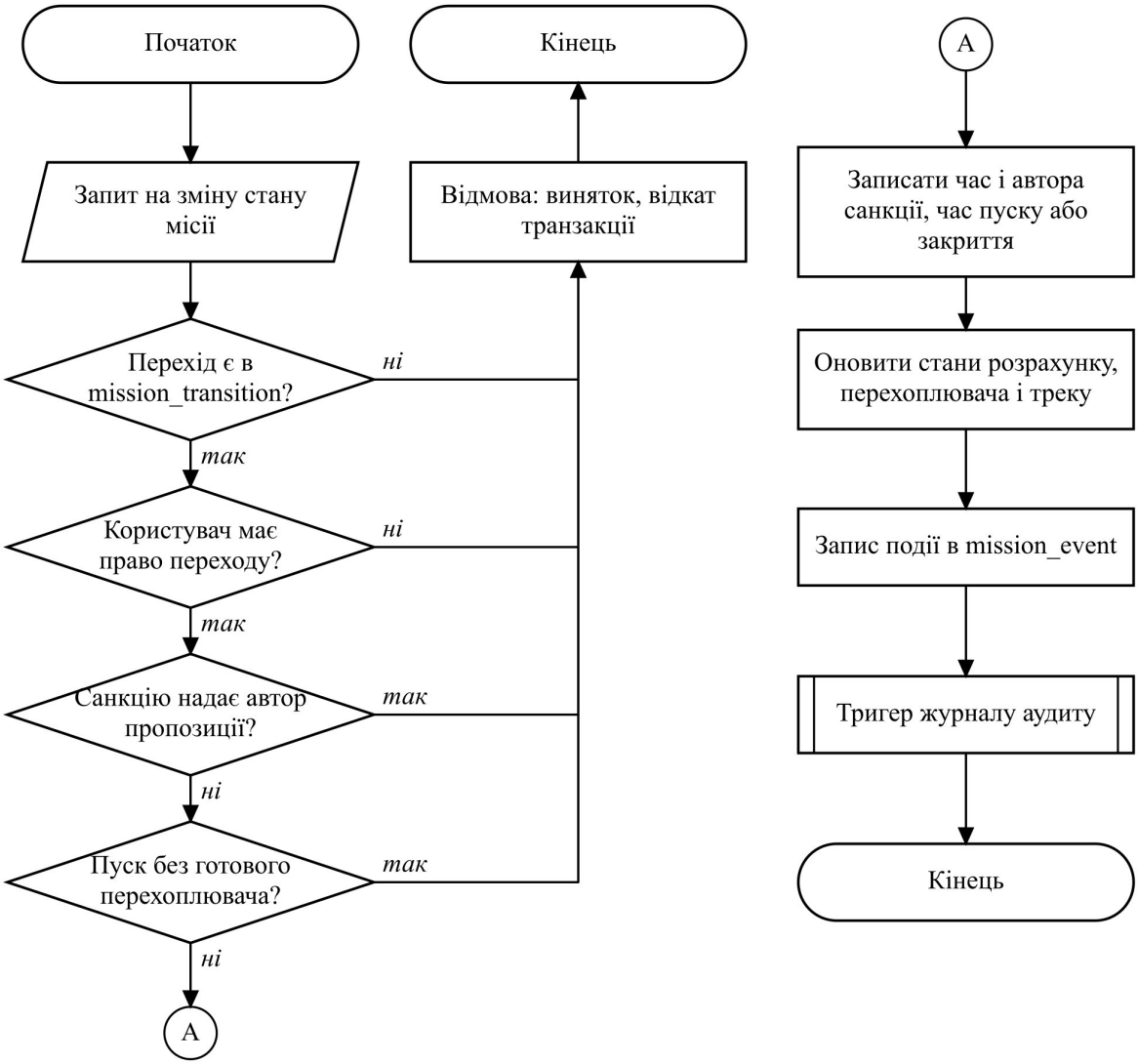


Рис. 2.4 — Блок-схема модуля керування місією

```
CREATE FUNCTION trg_mission_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_perm varchar;
    v_user uuid := fn_app_user();
BEGIN
    IF NEW.state = OLD.state THEN RETURN NEW; END IF;
    SELECT perm INTO v_perm FROM mission_transition
    WHERE from_state = OLD.state AND to_state = NEW.state;
    IF NOT FOUND THEN                      -- перехід поза діаграмою станів
        RAISE EXCEPTION 'Недопустимий перехід стану місії: «%» → «%»',
              OLD.state, NEW.state USING ERRCODE = 'P0004';
    END IF;
    IF v_user IS NOT NULL AND NOT fn_has_perm(v_user, v_perm) THEN
        RAISE EXCEPTION 'Немає права % для переходу «%» → «%»',
              v_perm, OLD.state, NEW.state USING ERRCODE = '42501';
    END IF;
    ...
END $$;
```

Рис. 2.5 — Фрагмент тригерної функції trg_mission_guard

6.5. Модуль журналу аудиту

Модуль журналу аудиту (рис. 2.6) складається з двох алгоритмів. Тригер trg_audit на таблицях mission, personnel, role_permission і engagement_report формує стан запису до і після зміни, отримує блокування для послідовного запису, читає хеш попереднього запису й обчислює SHA-256 від нього разом із часом, суб’єктом, дією та даними. Функція fn_audit_verify проходить журнал за зростанням номера, перераховує хеші й рахує порушення. Оскільки кожен хеш залежить від попереднього, зміна будь-якого запису розриває ланцюжок і виявляється перевіркою.

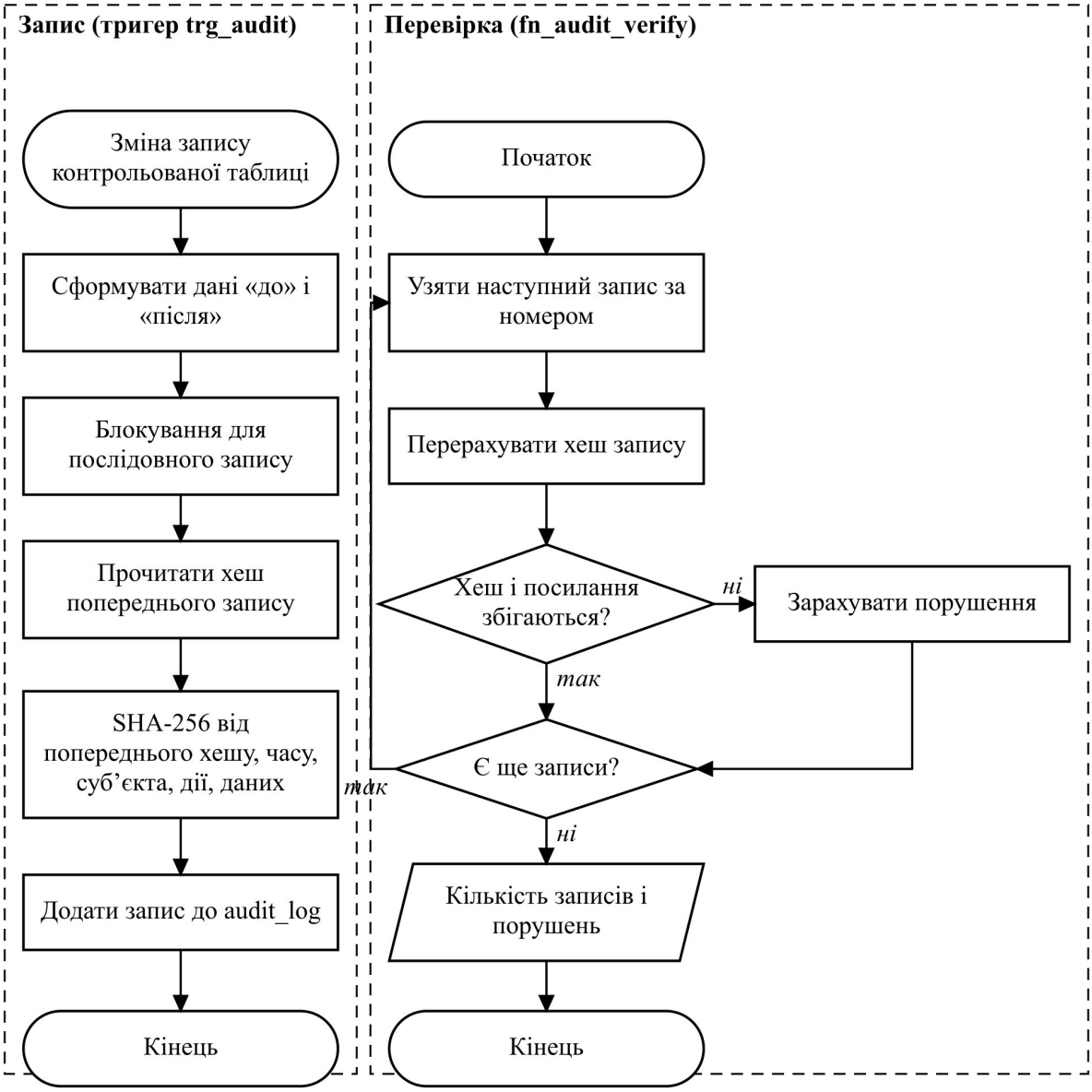


Рис. 2.6 — Блок-схема модуля журналу аудиту

7. ОПИС ІНТЕРФЕЙСУ ПРОГРАМНОГО ПРОДУКТУ

Інтерфейс АРМ — веб-сторінки з однаковою структурою: верхня панель з назвою системи, меню розділів і даними користувача (ім’я, код і назва ролі, позивний розрахунку), область повідомлень і робоча область. Меню формується за правами ролі: користувач бачить лише ті розділи, на які має право. Нижче описано сценарій роботи, пройдений на тестовому стенді.

7.1. Вхід до системи

Сторінка входу (рис. 2.7) у навчальному стенді імітує єдиний вхід з двофакторною автентифікацією: користувач обирає обліковий запис зі списку, згрупованого за ролями. Після входу відкривається перший розділ, на який користувач має право.

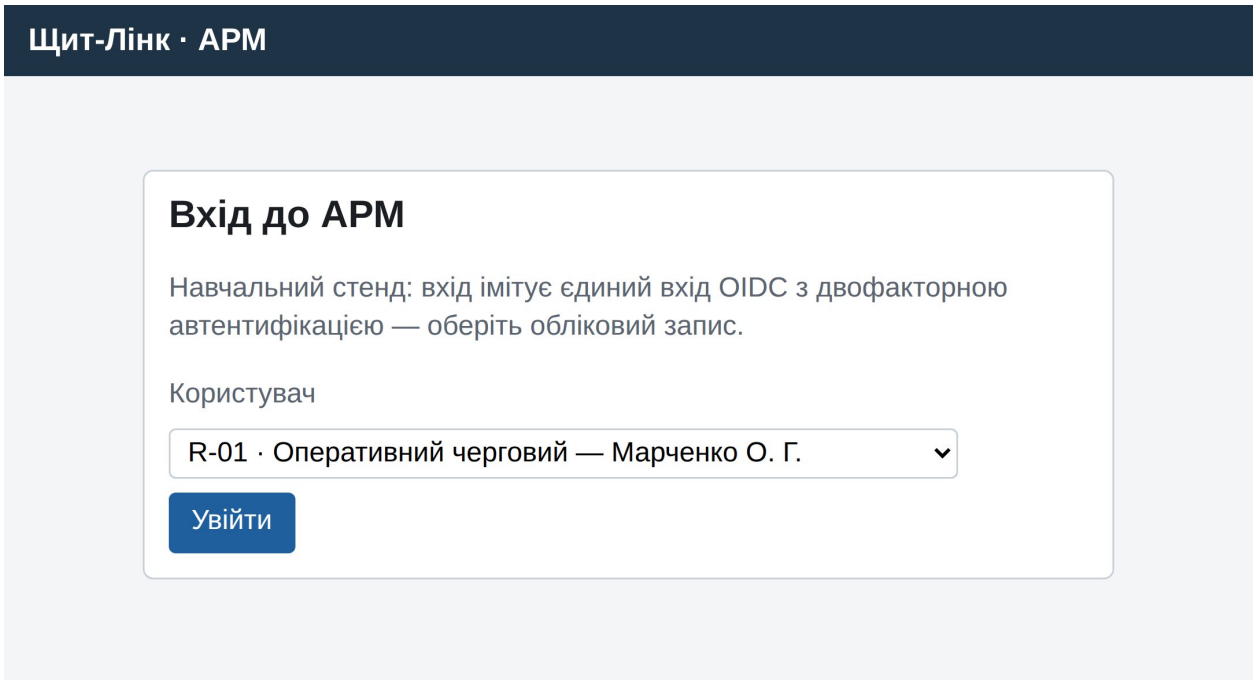


Рис. 2.7 — Сторінка входу до АРМ

7.2. Повітряна обстановка

Сторінка «Обстановка» (рис. 2.8) — головний екран аналітика, офіцера бойового управління та оперативного чергового. Чотири картки показують кількість активних треків, місій у роботі, готових розрахунків і сенсорів у мережі. Таблиця черги загроз упорядкована за оцінкою загрози (смуга й

число), ворожі цілі виділено червоним; для кожного треку видно час підльоту, об’єкт під загрозою та активну місію.

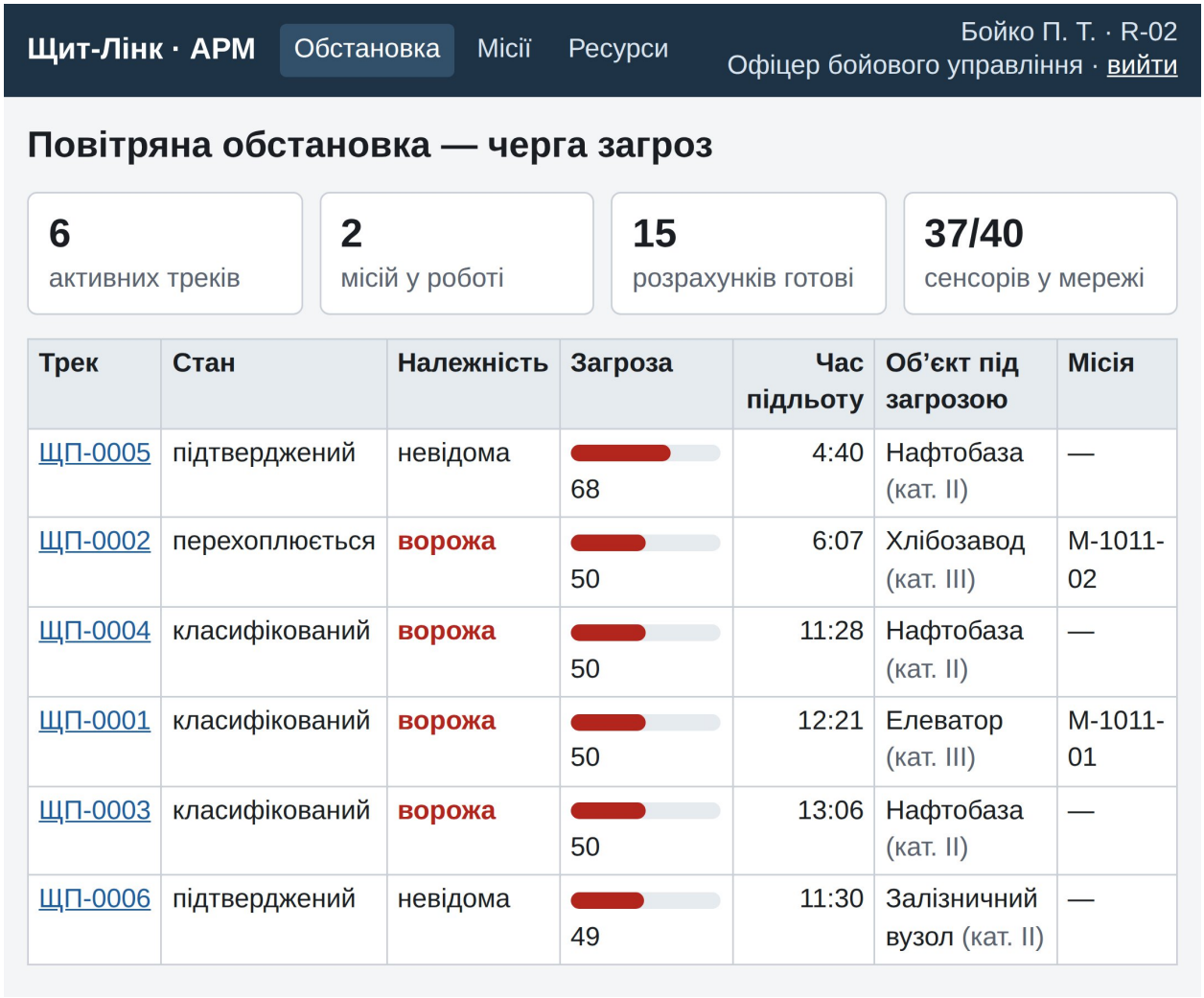


Рис. 2.8 — Сторінка «Обстановка»: черга загроз

Номер треку відкриває картку треку (рис. 2.9): належність, оцінка загрози, остання позиція, хто класифікував, останні виявлення від різних сенсорів. Аналітик бачить форму класифікації, офіцер бойового управління — список розрахунків-кандидатів, упорядкованих за досяжністю й відстанню, і кнопку «Запропонувати місію».

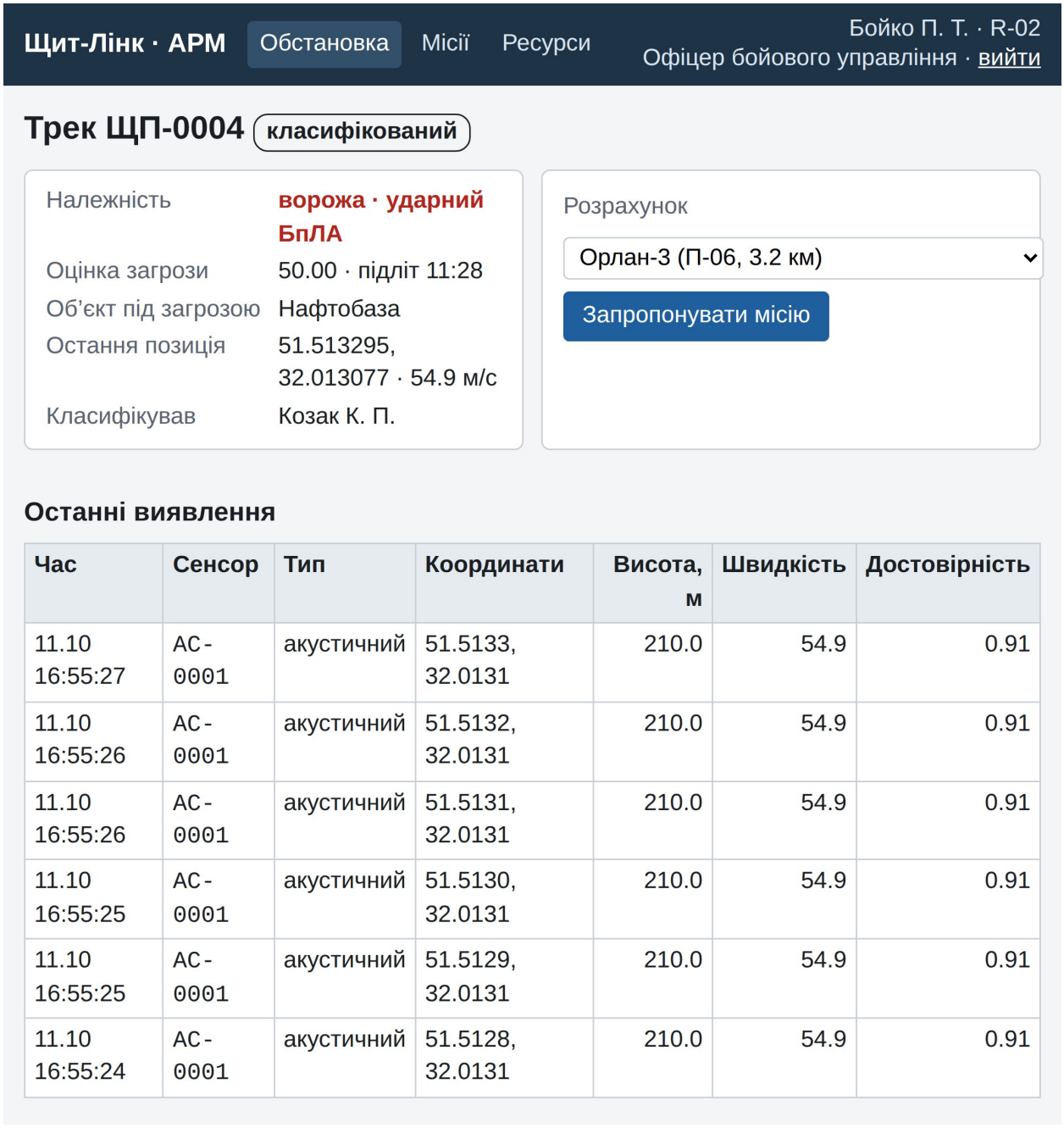


Рис. 2.9 — Картка треку з пропозицією місії

7.3. Життєвий цикл місії

Після пропозиції відкривається картка місії у стані «запропонована» (рис. 2.10). Офіцер бойового управління не має права санкції, тому дій для нього немає — це наочний прояв правила двох осіб.

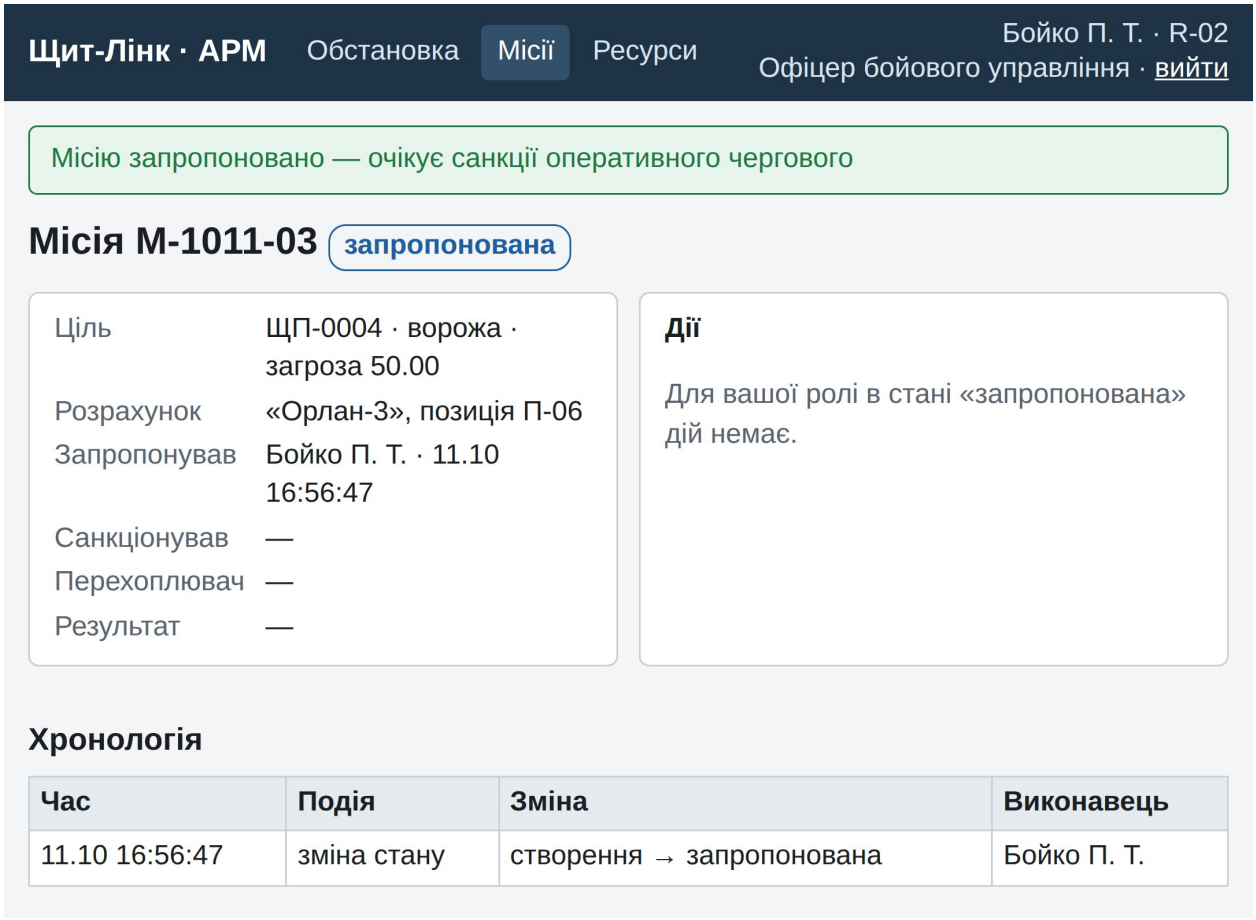


Рис. 2.10 — Картка місії після пропозиції

Оперативний черговий відкриває ту саму місію й натискає «Санкціонувати» (рис. 2.11); у картці з’являються його ім’я та час санкції, а в хронології — нова подія. Перелік місій з фільтром за станом наведено на рис. 2.12.

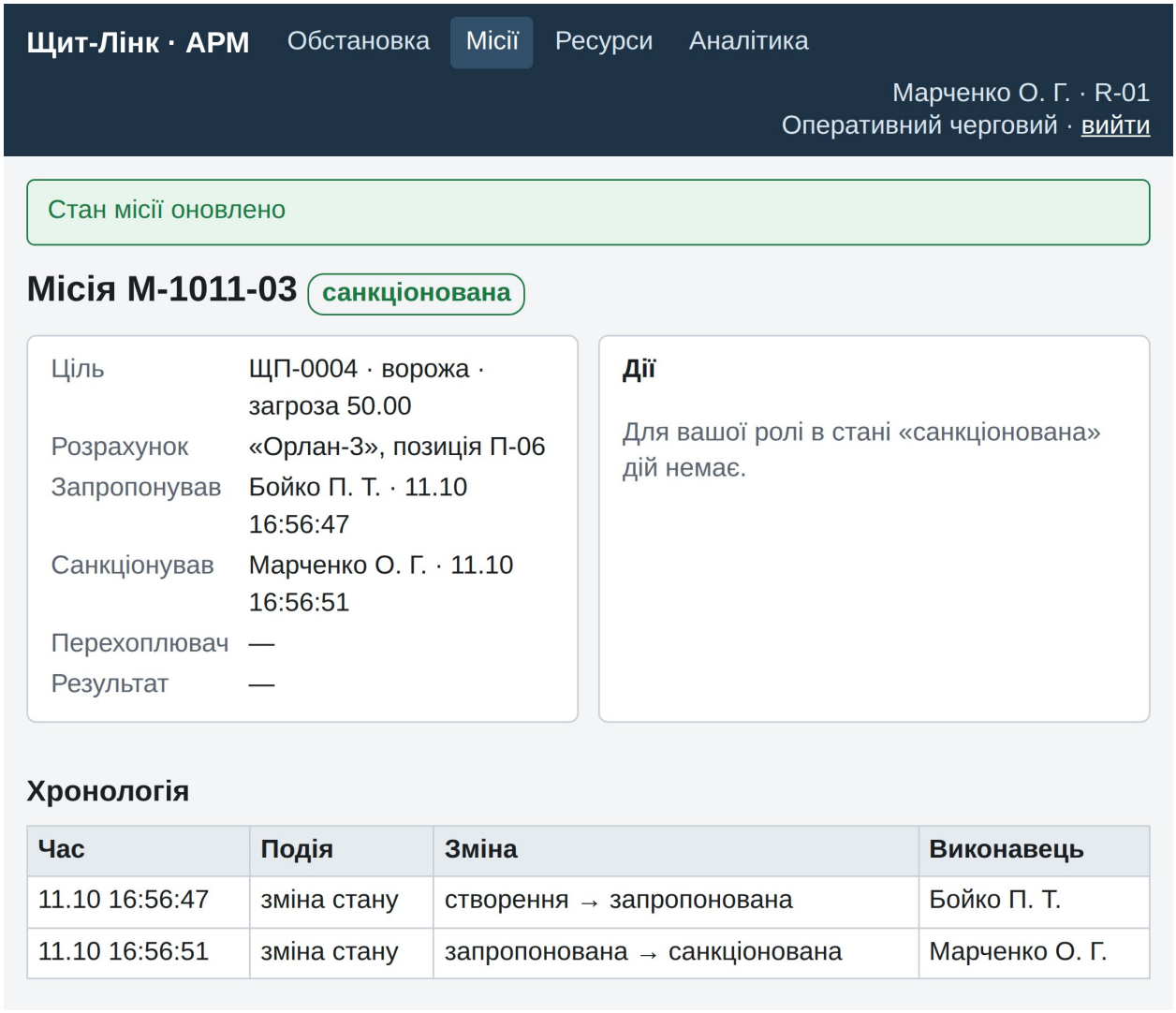


Рис. 2.11 — Картка місії після санкції оперативного чергового

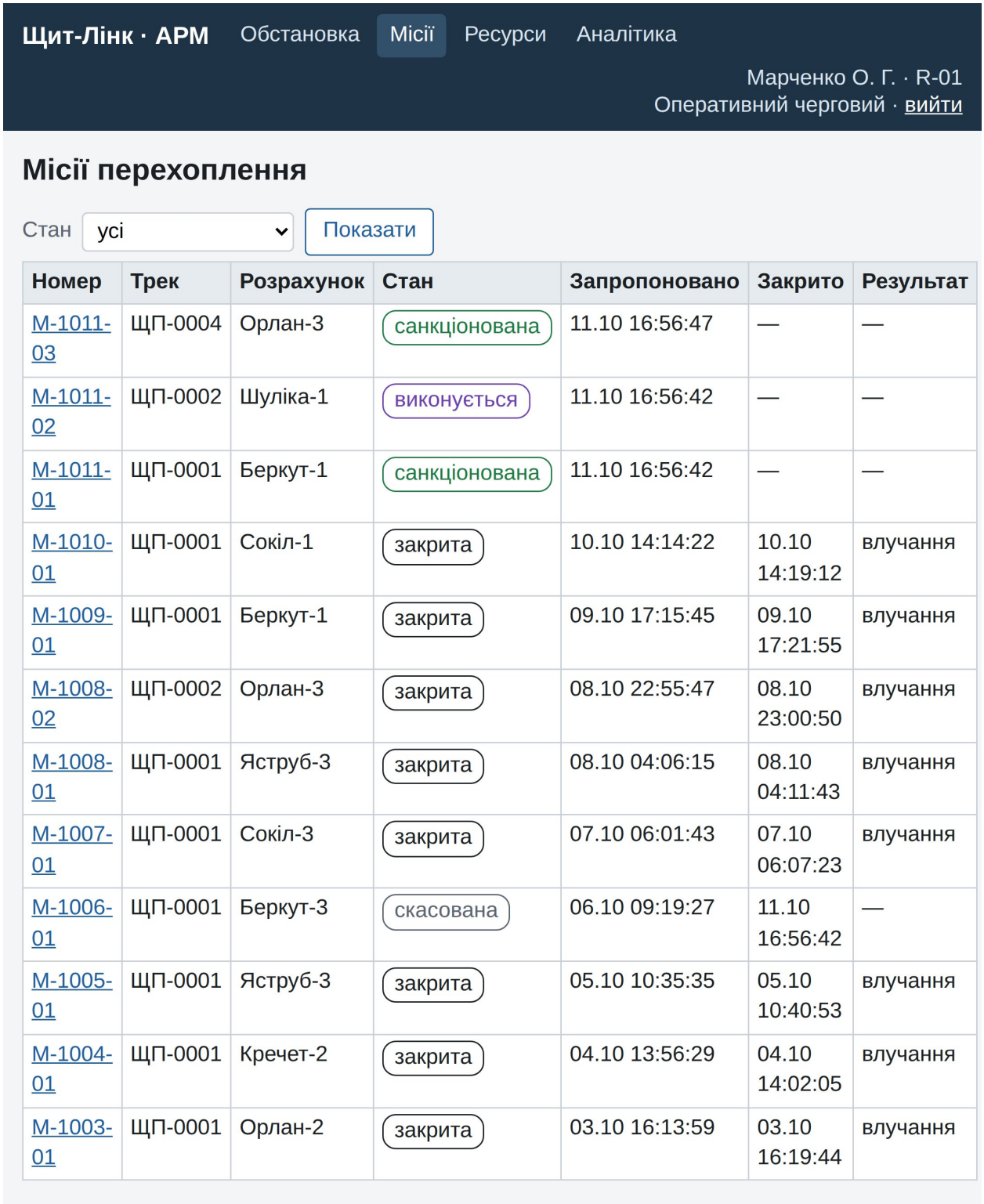


Рис. 2.12 — Перелік місій перехоплення

Командир призначеного розрахунку приймає завдання (рис. 2.13); для пуску він обирає готовий перехоплювач своєї позиції зі списку. Після пуску й доповіді про результат місія закривається (рис. 2.14): хронологія містить усі п’ять змін стану з виконавцями, а перехоплювач і трек отримують нові стани автоматично.

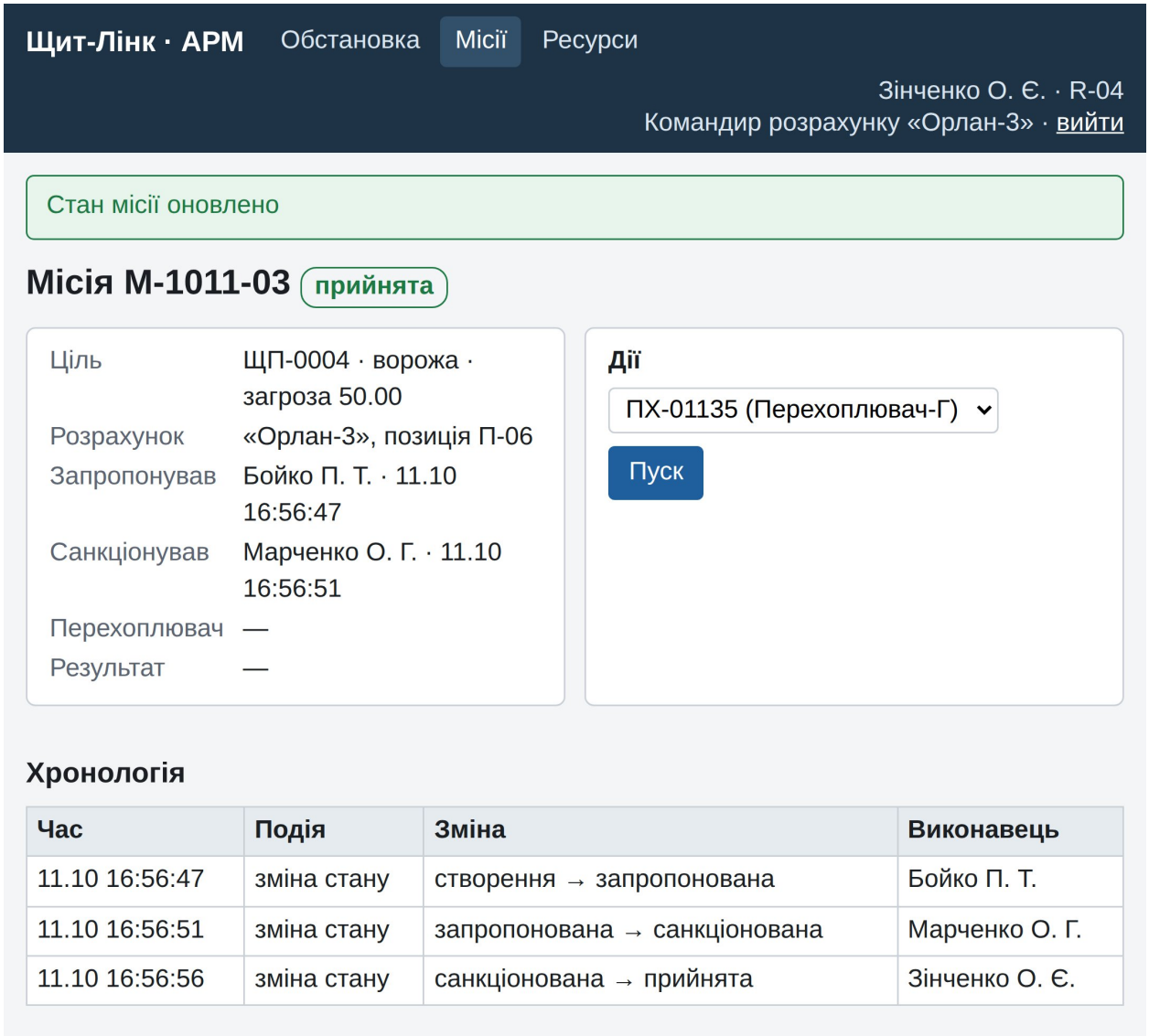


Рис. 2.13 — Картка місії, прийнятої розрахунком

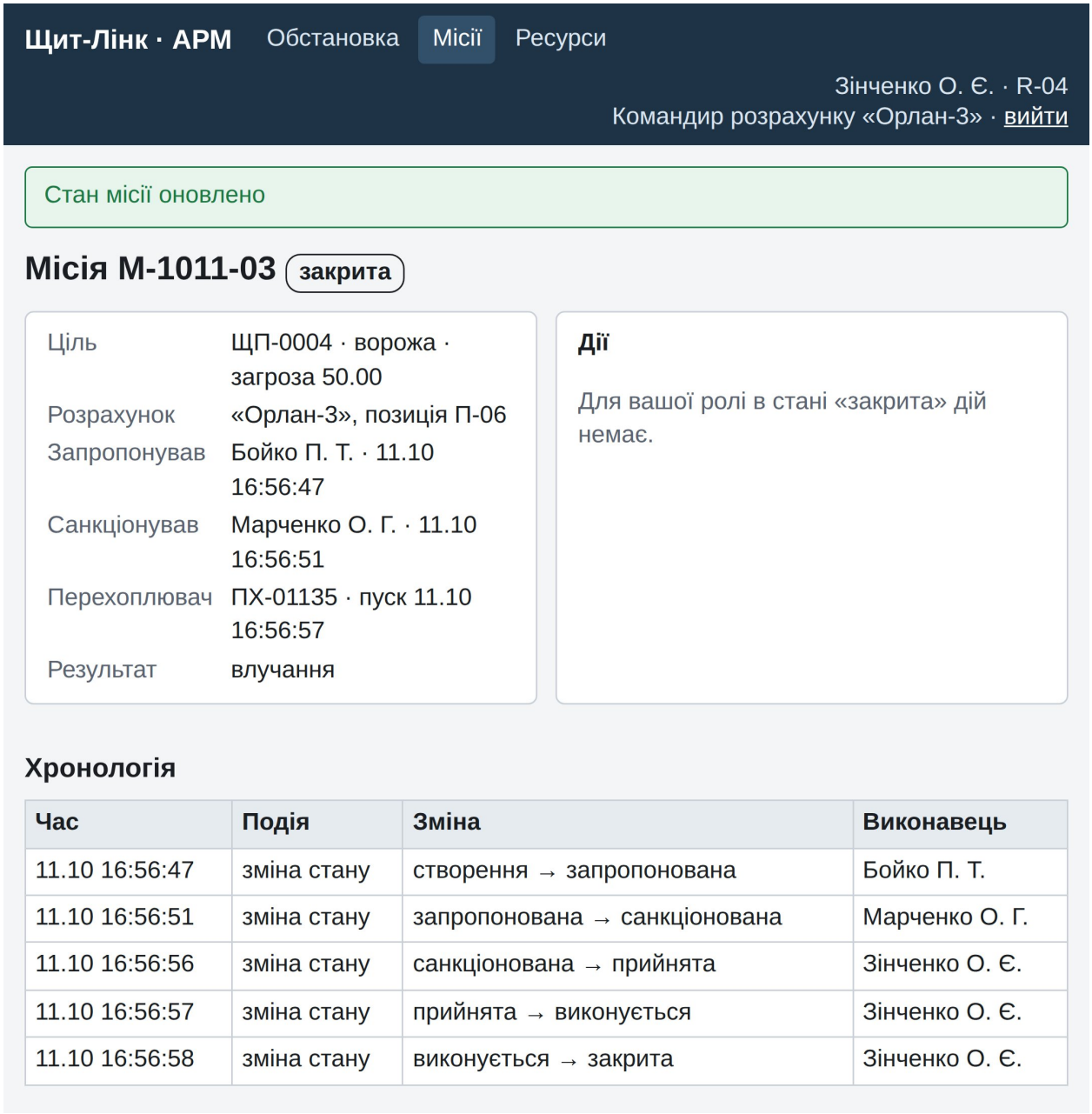


Рис. 2.14 — Закрита місія М-1011-03 з повною хронологією

7.4. Ресурси

Сторінка «Ресурси» (рис. 2.15) показує готовність за позиціями й перелік перехоплювачів обраної позиції. Логіст може додати перехоплювач, змінити його дані (рис. 2.16) або видалити. Спроба видалити перехоплювач, уже використаний у місіях, відхиляється обмеженням зовнішнього ключа, і користувач бачить пояснення (рис. 2.17).

Щит-Лінк · АРМ Ресурси

Савчук П. О. · R-08
Логіст · вийти

Готовність вогневих ресурсів

Розрахунки: готові / на завданні / всього; перехоплювачі: готові / всього.

Позиція	Сектор	Стан	Розрахунки	Перехоплювачі
П-01	Сектор 1	активна	2 / 0 / 2	1 / 6
П-02	Сектор 5	активна	2 / 0 / 2	4 / 6
П-03	Сектор 7	активна	2 / 0 / 2	1 / 6
П-04	Сектор 7	активна	2 / 0 / 2	4 / 6
П-05	Сектор 7	активна	2 / 0 / 2	2 / 6
П-06	Сектор 7	активна	1 / 1 / 2	2 / 6
П-07	Сектор 2	активна	1 / 0 / 2	3 / 5
П-08	Сектор 7	активна	1 / 0 / 2	4 / 5
П-09	Сектор 2	активна	1 / 0 / 1	5 / 5
П-10	Сектор 7	передислокація	1 / 0 / 1	4 / 5

Перехоплювачі позиції П-01 + Додати

Серійний номер	Модель	Позиція	Стан	Вильоти	Прошивка	Дії
ПХ-01100	Перехоплювач-А	П-01	застосований	1	4.1.8	змінити видалити
ПХ-01110	Перехоплювач-В	П-01	застосований	1	4.3.4	змінити видалити
ПХ-01120	Перехоплювач-А	П-01	застосований	1	4.1.7	змінити видалити
ПХ-01130	Перехоплювач-В	П-01	застосований	1	4.2.8	змінити видалити

Рис. 2.15 — Сторінка «Ресурси»: готовність позицій і перехоплювачі

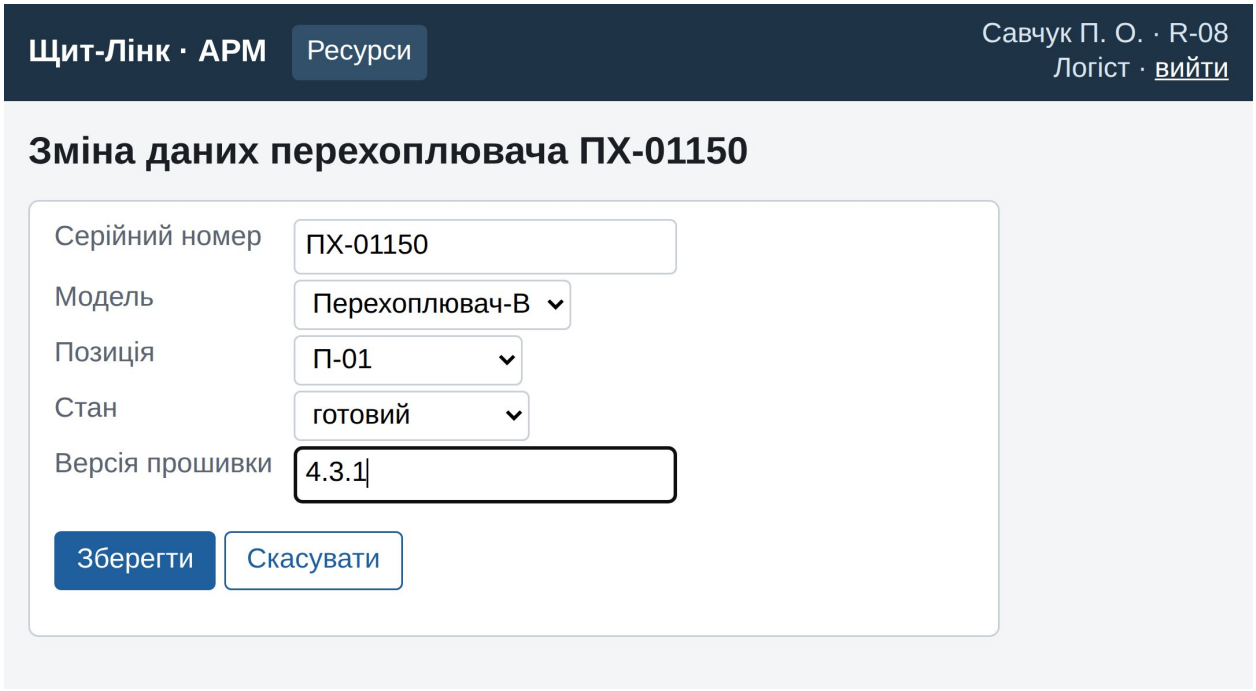


Рис. 2.16 — Форма зміни даних перехоплювача

Щит-Лінк · АРМ Ресурси Савчук П. О. · R-08 Логіст · вийти

Неможливо видалити: перехоплювач використано в місіях (обмеження зовнішнього ключа)

Готовність вогневих ресурсів

Розрахунки: готові / на завданні / всього; перехоплювачі: готові / всього.

Позиція	Сектор	Стан	Розрахунки	Перехоплювачі
П-01	Сектор 1	активна	2 / 0 / 2	1 / 6
П-02	Сектор 5	активна	2 / 0 / 2	4 / 6
П-03	Сектор 7	активна	2 / 0 / 2	1 / 6
П-04	Сектор 7	активна	2 / 0 / 2	4 / 6
П-05	Сектор 7	активна	2 / 0 / 2	2 / 6
П-06	Сектор 7	активна	1 / 1 / 2	2 / 6
П-07	Сектор 2	активна	1 / 0 / 2	3 / 5
П-08	Сектор 7	активна	1 / 0 / 2	4 / 5
П-09	Сектор 2	активна	1 / 0 / 1	5 / 5
П-10	Сектор 7	передислокація	1 / 0 / 1	4 / 5

Перехоплювачі позиції П-01 + Додати

Серійний номер	Модель	Позиція	Стан	Вильоти	Прошивка	Дії
ПХ-01100	Перехоплювач-А	П-01	застосований	1	4.1.8	змінити видалити
ПХ-01110	Перехоплювач-В	П-01	застосований	1	4.3.4	змінити видалити
ПХ-01120	Перехоплювач-А	П-01	застосований	1	4.1.7	змінити видалити
ПХ-01130	Перехоплювач-В	П-01	застосований	1	4.2.8	змінити видалити

Рис. 2.17 — Відмова у видаленні використаного перехоплювача

7.5. Аналітика та журнал аудиту

Сторінка «Аналітика» (рис. 2.18) для обраного періоду показує кількість місій, частку влучань, середній час від пропозиції до санкції і від санкції до

пуску, розподіл результатів і рейтинг розрахунків. Сторінка «Аудит» (рис. 2.19) доступна лише офіцеру кібербезпеки: вона перевіряє ланцюжок хешів і показує останні записи журналу.

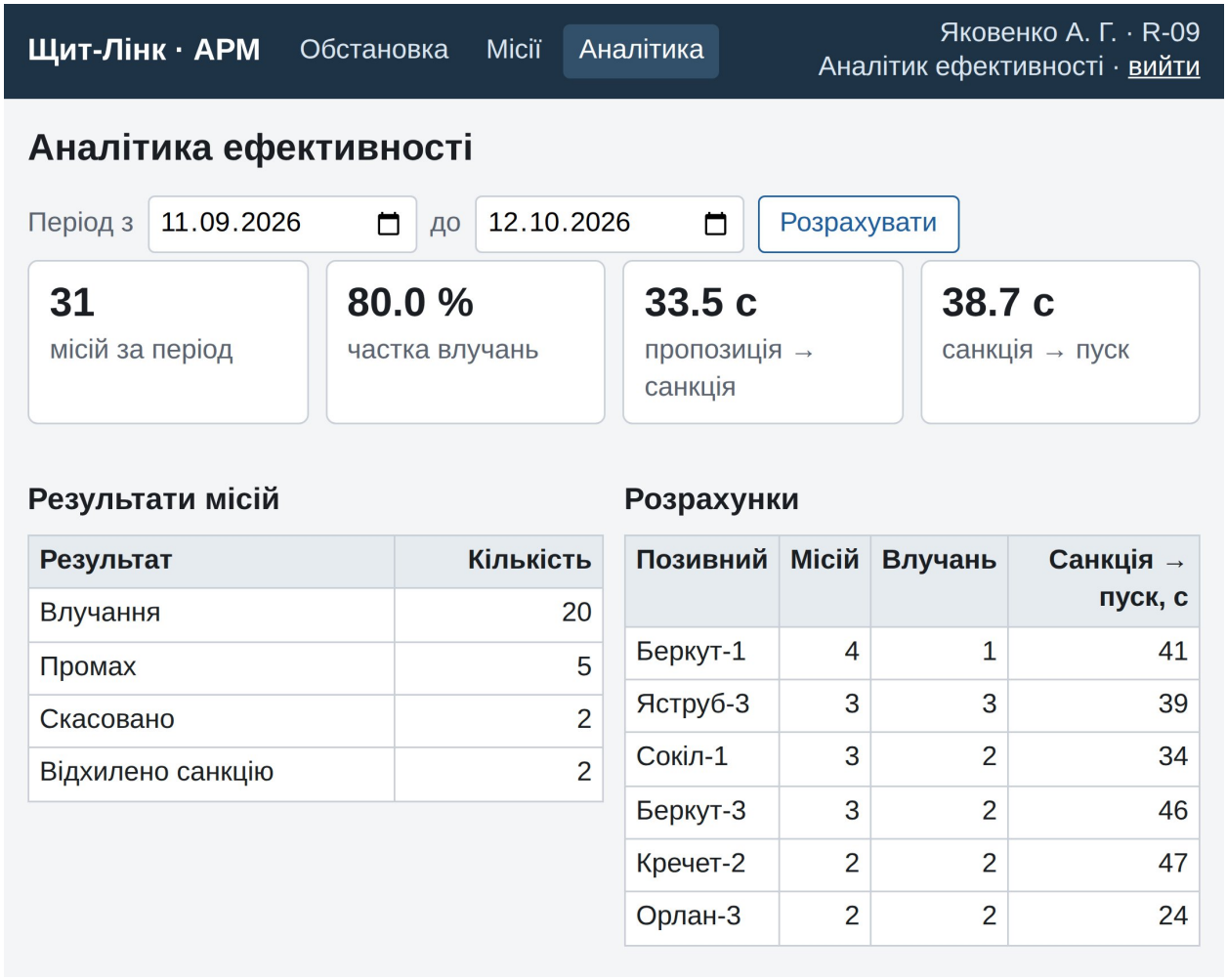


Рис. 2.18 — Сторінка «Аналітика»

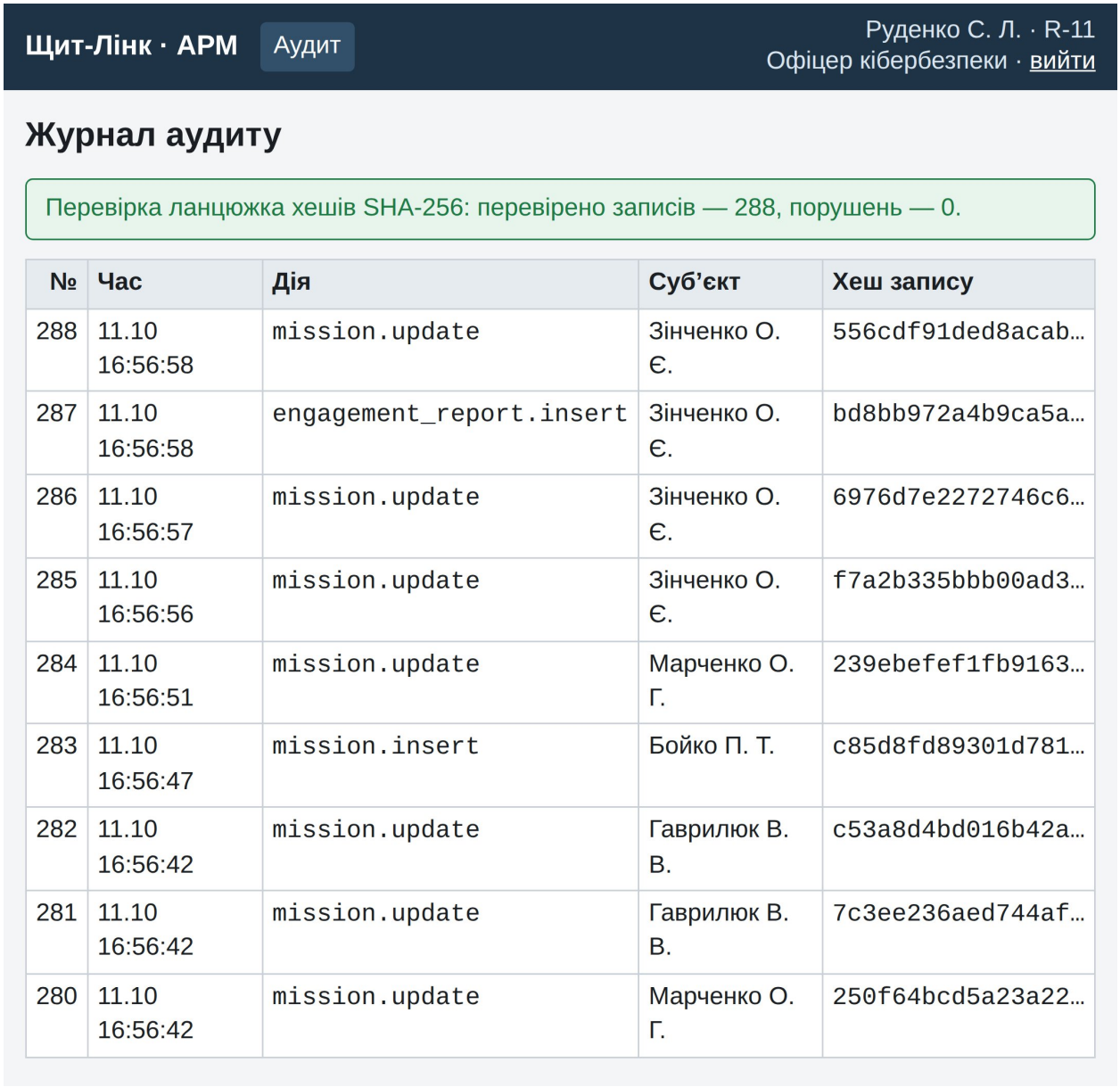


Рис. 2.19 — Сторінка «Аудит» з перевіркою ланцюжка хешів

7.6. Програмний інтерфейс для шлюзів

Периферійні шлюзи працюють з продуктом через REST API: POST /api/v1/detections приймає виявлення, GET /api/v1/threats повертає чергу загроз у форматі JSON. Обмін для одного виявлення показано на рис. 2.20: сервер зіставив виявлення з існуючим треком і повернув його номер. Запит від сенсора «АС-0008», що перебуває поза мережею, отримує відповідь 422 з причиною, а запит з невірним ключем — 401.

```
POST /api/v1/detections HTTP/1.1
Content-Type: application/json
X-Gateway-Token: ********

{
  "sensor": "AC-0001",
  "detected_at": "2026-10-11T16:55:11.668+03:00",
  "lat": 51.510395,
  "lon": 32.013077,
  "alt": 210,
  "speed": 54.9,
  "bearing": 187.5,
  "signature": "ударний поршневий",
  "confidence": 0.91
}

HTTP/1.1 201 CREATED
{
  "detection_id": 652,
  "track_id": "8506430f-cb51-447f-97c4-c2d3fe05b0cb",
  "track_number": "ЩП-0004"
}
```

Рис. 2.20 — Обмін з REST API під час передавання виявлення

8. РЕАЛІЗАЦІЯ ФУНКЦІОНАЛЬНИХ ВИМОГ ДО БАЗИ ДАНИХ

У табл. 2.6 для кожної вимоги ФВ-БД з висновків ЛР1 наведено засоби реалізації та результат автоматичної перевірки на тестовому стенді.

Таблиця 2.6 — Реалізація та перевірка функціональних вимог до бази даних

Вимога	Засоби реалізації	Результат перевірки	Стан
ФВ-БД-01	schema.sql (ЛР1) + db/impl.sql	19 таблиць, 53 функції, 9 тригерів, 2 представлення створено без помилок	виконано
ФВ-БД-02	db/seed.py через функції й тригери	основні таблиці — від 10 записів, операційні — від 24 до 651	виконано
ФВ-БД-03	Сторінки «Ресурси», форма перехоплювача; обмеження FOREIGN KEY	створення, зміна, видалення виконано; видалення використаного перехоплювача відхилено	виконано
ФВ-БД-04	fn_ingest_detection, POST /api/v1/detections	відповідь 201 і трек ЩП-0004; сенсор поза мережею — 422; невірний ключ — 401	виконано
ФВ-БД-05	Представлення v_threat_queue, сторінка «Обстановка»	треки впорядковано за спаданням оцінки загрози	виконано
ФВ-БД-06	fn_propose_mission, переходи станів, CHECK approved_by ≠ proposed_by	місія М-1011-03 пройшла повний цикл через інтерфейс; санкцію автором відхилено	виконано
ФВ-БД-07	Таблиця mission_transition, тригери trg_mission_guard і trg_mission_after	недопустимий перехід і перехід без права відхилено; кожна зміна стану — подія	виконано
ФВ-БД-08	Тригер trg_report_after, UNIQUE (mission_id)	повторний звіт відхилено; сума вильотів дорівнює кількості пусків	виконано
ФВ-БД-09	Представлення v_readiness	рядок для кожної з 10 позицій	виконано
ФВ-БД-10	Функція fn_mission_stats, сторінка «Аналітика»	31 місія, частка влучань 80,0 %	виконано
ФВ-БД-11	Тригери trg_audit і audit_immutable, fn_audit_verify	ланцюжок 288 записів цілий; зміна й видалення заборонені; штучну підробку запису № 40 виявлено	виконано
ФВ-БД-12	Ролі shch_analyst, shch_officer, shch_crew, shch_admin, shch_security; GRANT	аналітику відмовлено у створенні місії, офіцеру — у читанні журналу; офіцер кібербезпеки журнал читає	виконано
ФВ-БД-13	pg_dump, pg_restore	копію 174 КБ відновлено в нову БД; кількість записів у всіх таблицях збігається	виконано

Окремо перевірено стійкість журналу аудиту до підробки. Від імені суперкористувача було вимкнено тригер незмінності й змінено поле даних одного запису. Повторна перевірка виявила порушення саме в цьому записі (№ 40): хеш, перерахований з даних, більше не збігається зі збереженим, а ланцюжок посилань розірвано. Отже, навіть адміністратор СКБД не може непомітно змінити історію рішень.

Статистику тестових даних за 30 діб наведено на сторінці «Аналітика»: 31 місія, 20 влучань, 5 промахів, 2 скасування і 2 відхилені санкції; середній час від пропозиції до санкції — 33,5 с, від санкції до пуску — 38,7 с.

ВИСНОВКИ

У лабораторній роботі описано програмну реалізацію IoT-платформи «Щит-Лінк».

1) Сформульовано мету програмного продукту — скорочення часу від виявлення цілі до призначення розрахунку та підвищення частки успішних перехоплень за рахунок автоматизації обстановки, контрольованого життєвого циклу місії з правилом двох осіб і незмінного журналу аудиту.

2) Визначено вісім груп вхідних і вісім груп вихідних даних з джерелами, каналами, отримувачами та відповідними таблицями й представленнями БД.

3) Сформульовано системні вимоги до сервера, АРМ, периферійного шлюзу й мережі; їх достатність підтверджено на стенді: обробка виявлення займає в медіані 7,8 мс за бюджету затримки 2 с.

4) Обґрунтовано вибір засобів: PostgreSQL з PL/pgSQL — для виконання правил предметної області всередині СКБД; Python і Flask — для компактного веб-застосунку без ORM; Jinja2 і psycopg — для захисту від міжсайтових сценаріїв і SQL-ін’єкцій; Puppeteer — для автоматизованої перевірки інтерфейсу.

5) Описано алгоритм роботи продукту та побудовано блок-схеми структури, загального алгоритму обробки запиту, модулів прийому виявлень, керування місією та журналу аудиту.

6) Описано інтерфейс АРМ на тринадцяти знімках екрана працюючої програми — від входу до аудиту — та REST API для шлюзів.

7) Реалізовано й автоматично перевірено всі тринадцять функціональних вимог до БД з лабораторної роботи 1.

Отриманий програмний продукт є працюючим ядром системи, придатним для подальшого розвитку: підключення реальних протоколів сенсорів і телеметрії, картографічного подання обстановки та розгортання в контейнерах.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Лабораторна робота 2. Опис програмної реалізації системи : завдання з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026.

2. Кравченко П. Опис бази даних : звіт з лабораторної роботи № 1 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 52 с.

3. Безносюк Д. IoT система забезпечення перевезень морським транспортом : кваліфікаційна робота бакалавра : спец. 126 «Інформаційні системи та технології» / керівник О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 101 с.

4. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

5. Кравченко П. Збір та документування вимог для програмного продукту : звіт з практичної роботи № 1 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 42 с.

6. Вимоги до програмного забезпечення: збір, аналіз, управління : лекція 2 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 9 слайдів.

7. Засоби та інструменти розробки ПЗ : лекція 7 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 17 слайдів.

8. 41.10. Trigger Functions. PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. URL: https://www.postgresql.org/docs/18/plpgsql-trigger.html (дата звернення: 11.10.2026).

9. 9.5. Binary String Functions and Operators. PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. URL:

https://www.postgresql.org/docs/18/functions-binarystring.html (дата звернення: 11.10.2026).

10. GRANT. PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. URL: https://www.postgresql.org/docs/18/sql-grant.html (дата звернення: 11.10.2026).

11. pg_dump. PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. URL: https://www.postgresql.org/docs/18/app-pgdump.html (дата звернення: 11.10.2026).

12. Flask Documentation (3.1.x) / Pallets. URL: https://flask.palletsprojects.com/en/stable/ (дата звернення: 11.10.2026).

13. Jinja Documentation (3.1.x) / Pallets. URL: https://jinja.palletsprojects.com/en/stable/ (дата звернення: 11.10.2026).

14. Psycopg 3 — PostgreSQL database adapter for Python. URL: https://www.psycopg.org/psycopg3/docs/ (дата звернення: 11.10.2026).

15. Puppeteer Documentation. URL: https://pptr.dev/ (дата звернення: 11.10.2026).

16. Проектування програмних систем : лекція 5 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 43 слайди.

17. ISO 5807:1985. Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts. Geneva : ISO, 1985. 25 p.

18. Кравченко П. Проектування системи : звіт з практичної роботи № 4 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 43 с.